

Mipmaps are too dark around transparent edges in textures (e.g. side of grass)

MC-114265 Rendering

Status	Resolved / Fixed
Affected	Minecraft 1.11.2, Minecraft 17w06a, Minecraft 1.13, Minecraft 1.13.1-pre1, Minecraft 1.14.2, Minecraft 1.14.3 Pre-Release 2 (+62 more)
Fixed in	25w41a
Reported	2017-02-28

STEPS TO REPRODUCE

1. Launch Minecraft with the settings from the provided `options.txt` file.
2. Open the provided `World.zip` world.
3. Enter the world containing a row of grass blocks placed on a row of dirt blocks.
4. Look straight ahead without moving.
5. Observe the side of the grass blocks for dark patches, especially where the level 3 mipmap or a more distant mipmap is used.

OBSERVED BEHAVIOUR

Actual behavior

When mipmaps are enabled, textures with transparent or partially transparent pixels develop dark, nearly black patches along their opaque edges at distances where lower-resolution mipmap levels are used. The issue is particularly visible on the side texture of grass blocks: around mipmap level 3, approximately half of the grass portion can appear black when viewed from the optimal distance.

The darkening occurs during mipmap generation. Color values near transparent pixels are averaged as though the transparent pixels were black, without correctly weighting the color contribution by alpha. As a result, the generated mipmap contains unnaturally dark edge pixels. The effect is amplified by the handling of alpha during the averaging process and is visible when the mipmapped texture is rendered in-game.

EXPECTED BEHAVIOUR

When mipmaps are enabled, textures with transparent edges, such as the side of a grass block, should retain their correct colors and brightness at all viewing distances. The grass should blend smoothly with surrounding dirt without developing abnormally dark or nearly black patches when lower-resolution mipmap levels are displayed.

ENVIRONMENT

Minecraft Java Edition Versions: 1.11.2, 17w06a, 1.13, 1.13.1-pre1, 1.14.2, 1.14.3 Pre-Release 2, 1.14.3, 1.14.4 Pre-Release 6, 1.14.4, 1.15 Pre-release 1, 1.15.2, 1.16.1, 1.16.5, 21w10a, 1.17, 1.17.1, 21w37a, 21w38a, 21w42a, 21w43a, 1.18 Pre-release 1, 1.18 Pre-release 7, 1.18 Pre-release 8, 1.18 Release Candidate 3, 1.18, 1.18.1 Release Candidate 1, 1.18.1, 22w03a, 22w05a, 1.18.2, 22w13a, 22w15a, 22w18a, 22w19a, 1.19 Pre-release 1, 1.19 Pre-release 2, 1.19 Pre-release 3, 1.19 Pre-release 4, 1.19, 1.19.1 Pre-release 5, 1.19.1, 1.19.2, 22w44a, 22w46a, 1.19.3 Pre-release 1, 1.19.3 Pre-release 2, 1.19.3, 23w03a, 23w04a, 23w05a, 1.19.4 Pre-release 1, 1.19.4, 23w12a, 23w13a, 1.20.1, 23w33a, 24w13a, 24w14a, 1.20.6, 24w36a, 1.21.3, 1.21.4 Release Candidate 3, 25w02a, 1.21.5, 25w16a, 1.21.6 Pre-Release 3, 1.21.6 Pre-Release 4, 1.21.6 (includes snapshots)

ORIGINAL DESCRIPTION

The bug

This has been an issue for a while. It's especially noticeable at level 3 of the side of grass. See the screenshots, where I placed a row of grass on a row of dirt. With mipmaps, half of the grass block is practically black at the 'optimal' distance where the 2x2 mipmap or further is used.

How to reproduce

Open this world

World.zip

with these settings

options.txt

and look straight ahead without moving.

Observe the dark patches with the side of the grass.

Proposed solution

Using Yarn mappings for 1.14.4, replace this method:

```
net.minecraft.client.texture.Sprite  
private static int blendPixels(int int_1, int int_2, int int_3, int int_4) {  
return SRGBAverager.average(int_1, int_2, int_3, int_4);  
}
```

using these classes provided by accountid:unknown:

SRGBAverager.java

SRGBCalculator.java

SRGBTable.java

Original analysis

I've looked it up in MCP 9.30/9.31 (corresponding to Minecraft 1.10.2). In `net.minecraft.client.renderer.texture.TextureUtil`, `generateMipmapData` is responsible for mipmap generation. It turns out not to use proper weighting by the alpha channel. Anything that isn't fully transparent is given the full weight, and anything that is is treated as black. That's where the darkness comes from.

The correct model is most natural when using colors premultiplied by alpha: just average everything. If you don't use premultiplied colors, you must multiply colors by alpha before averaging and divide the resulting color by the resulting alpha afterward.

Premultiplied also has the advantage that any blending steps you don't control directly work correctly, like texture filtering and frame buffer blending. Minecraft won't really experience either the advantages or the disadvantages, but it's the cleaner model that would've prevented problems like this bug from the start. That's something to consider.

Adding to the problem is that Minecraft tries to apply gamma correction to the alpha channel. That blows up the opacity, only emphasizing exactly those texels that are too dark. Gamma correction of the alpha channel doesn't make sense anyway because colors can deviate either way depending on what's blended with what. The genera

MANUAL RESULT

Version used	
Reproduced?	YES / NO / PARTIAL
Crash file	
Notes	
Tested by / date	

Live issue: <https://bugs.mojang.com/browse/MC/issues/MC-114265>